

Analyses: Classification Performance

Description

This file enables post-hoc analyses of the performance values (F1-scores, accuracies, p-values) of the KNN and MLP classifications. It provides density distributions for these performance metrics.

Preliminaries: Performance Metrics

The performance metrics are derived by using the `confusionMatrix()` function of the `caret` package. See the code files in `analyses_Classification_KNN.Rmd` and `analyses_Classification_MLP.Rmd` for the actual implementation. In the case of a two class classification problem (e.g. Aurignacien vs. Uruk V), a confusion matrix might look as in the example below.

```
knitr::include_graphics("~/Github/PaleoSigns/figures_external/confusionMatrix.png")
```

Predicted	Reference	
	Event	No Event
Event	A	B
No Event	C	D

Figure 1: Table. Confusion matrix as given in the `confusionMatrix()` documentation.

The accuracy is then calculated as

$$accuracy = \frac{A + D}{A + D + B + C}. \quad (1)$$

The function also provides the so-called “no information rate” baseline, which corresponds to the accuracy achieved by simply always choosing the most frequent class label in the test set. For example, for the classification of Aurignacien vs. Uruk V Proto-Cuneiform, there are 101 sequences in the test set overall, of which 69 are Aurignacien sequences, and 32 are Uruk V Proto-Cuneiform sequences (examples of predictions of the last model run with the observed values are given in the folder `results/classifier/MLP/predictions`). Thus, the baseline in this case is

$$accuracy.baseline = \frac{69}{101} = 0.6831683. \quad (2)$$

Note that this baseline is marked with a red vertical line in the accuracy plots given below. Moreover, the `confusionMatrix()` function provides a p-value for a one-sided binomial test (via `binom.test`) for whether the actual accuracy value of the model is significantly higher than the baseline (cf. Kuhn, 2008, p. 15). Assuming a binomial distribution for random draws of two outcomes (e.g. 0 and 1 typically, or class labels UrukV and Aurignacien in our case) we have the null hypothesis and alternative hypothesis given as

$$H_0 : p = p_0 \quad (3)$$

and

$$H_1 : p > p_0 \quad (4)$$

respectively (cf. Hollander et al., 2014, pp. 11). Where p_0 in our case represents the “no information rate” baseline, and p is the probability (in this case relative frequency) with which a given model identifies the correct class label. In the example given earlier, we have 101 sequences, i.e. $n = 101$ trials of classification. The expected value (mean) of successes (here termed B) under the null hypothesis ($p_0 = 0.6831683$) is

$$E_{p_0}(B) = n \times p_0 = 101 \times 0.6831683 = 69, \quad (5)$$

and the variance is

$$\sigma_{p_0}^2(B) = n \times p_0 \times (1 - p_0) = 101 \times 0.6831683 \times 0.3168317 = 21.86139. \quad (6)$$

The standardized version of B is then

$$B^* = \frac{B - E_{p_0}(B)}{\sqrt{\sigma_{p_0}^2(B)}}. \quad (7)$$

Under H_0 the standardized B^* is assumed to follow a normal binomial distribution $\mathcal{N}(0, 1)$. We can then identify a z-score given an α -level of our choice, which the actual B^* we calculate from the data would have to surpass to reject H_0 . With the common $\alpha = 0.05$, we have $z_\alpha = 1.645$.

As a worked example, assume that in one of the runs of the MLP architecture classifying Aurignacien and Uruk V sequences the success rate is 75. In this case, we have:

$$B^* = \frac{B - E_{p_0}(B)}{\sqrt{\sigma_{p_0}^2(B)}} = \frac{75 - 69}{\sqrt{21.86139}} \sim 1.28 \quad (8)$$

Hence, $B^* < z_\alpha$, and we would accept H_0 , meaning that the MLP classification is not significantly better than the “no information rate” baseline. The model has not learned (from the training set) to distinguish between these sequences better than the baseline (i.e. if it just chose the most frequent label).

Apart from the accuracy, we also report the F_β -score, which can be generally defined as:

$$F_\beta = \frac{(1 + \beta^2) \times precision \times recall}{(\beta^2 \times precision) + recall}. \quad (9)$$

In the confusionMatrix() function the special case $\beta^2 = 1$ is implemented, thus giving an F_1 -score as

$$F_1 = \frac{2 \times precision \times recall}{precision + recall}. \quad (10)$$

Furthermore, precision is defined as

$$precision = \frac{A}{A + B}, \quad (11)$$

and recall as

$$recall = \frac{A}{A + C}. \quad (12)$$

In other words, precision measures how often the positive class (also called “Event” in the documentation) is correctly predicted (true positives) in relation to the overall number of times this class is predicted by the model (true positives + false positives). In other words, how often is the classifier correct when it predicts the positive class? The recall, on the other hand, measures how often the positive class is correctly predicted (true positives) in relation to the overall number of “actually observed” instances of this class in the test set (true positives + false negatives). In other words, how often does the model “find” or “recall” the correct class?

The F-score is then a harmonized mean over prediction and recall. The rational behind this is to make the results more robust in the face of imbalances in terms of the distribution of class labels in the test set.

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"
## [1] "x86_64-pc-linux-gnu"
```

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`.

```
library(ggplot2)
library(ggExtra)
library(gridExtra)
library(ggpubr)
library(ggribes)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v lubridate  1.9.3      v tibble    3.2.1
## v purrr      1.0.2      v tidyr     1.3.1
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::combine() masks gridExtra::combine()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(data.table)
```

```
##
```

```
## Attaching package: 'data.table'
```

```
##
```

```
## The following objects are masked from 'package:lubridate':
```

```
##
```

```
##      hour, isoweek, mday, minute, month, quarter, second, wday, week,
```

```
##      yday, year
##
## The following objects are masked from 'package:dplyr':
##
##      between, first, last
##
## The following object is masked from 'package:purrr':
##
##      transpose
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##      lift
```

Give the package versions.

```
##      caret      lattice data.table lubridate      forcats      stringr      dplyr
## "6.0-94" "0.22-5" "1.15.4" "1.9.3" "1.0.0" "1.5.1" "1.1.4"
##      purrr      readr      tidyr      tibble tidyverse ggribges ggpubr
## "1.0.2" "2.1.5" "1.3.1" "3.2.1" "2.0.0" "0.5.6" "0.6.0"
## gridExtra ggExtra ggplot2
## "2.3" "0.10.1" "3.5.1"
```

Multi-layer perceptron (MLP)

Load Data

```
# list files in the performance folders
files.mlp.performance <-
  list.files(path = "~/Github/PaleoSigns/results/classifier/MLP/performance", full.names = T)
```

Preprocessing

P-value adjustment for multiple comparisons: In case of multiple testing, we need account for the fact that the likelihood of finding a significant result by chance increases with the number of statistical tests executed. One of the most conservative methods to account for this is the so-called Bonferroni correction, i.e. multiplying the p-values with the number of tests. This method is appropriate when tests are independent of one another (cf. MacDonald 2014, p. 254-260). In our case, each test for each sequence can be seen as an independent trial. We therefore apply this conservative Bonferroni method (see also the descriptions in the vignette invoked by the command “`?p.adjust()`”).

```
# initialize empty data frame
performance.mlp.combined <- data.frame(hidden.structure = character(0),
                                       hidden.size = numeric(0),
                                       hidden.depth = numeric(0),
                                       err.fct = character(0),
                                       act.fct.name = character(0),
```

```

        algorithm = character(0),
        baseline = numeric(0),
        accuracy = numeric(0),
        accuracy.pvalue = numeric(0),
        precision = numeric(0),
        recall = numeric(0),
        f1 = numeric(0),
        accuracy.pvalue.corrected = numeric(0),
        comparison = character(0))

# loop through all files and combine them
for (file in files.mlp.performance){
  performance.data <- read.csv(file)
  # apply bonferroni correction
  performance.data$accuracy.pvalue.corrected <- p.adjust(performance.data$accuracy.pvalue,
    method = "bonferroni",
    n = nrow(performance.data))
  # get the subcorpora involved in the binary classification
  filename.parts <- unlist(strsplit(basename(file), "\\_|\\."))
  subcorpus1 <- filename.parts[3]
  subcorpus2 <- filename.parts[4]
  performance.data$comparison <- rep(paste(subcorpus1, "vs.", subcorpus2),
    nrow(performance.data))

  # add to data frame
  performance.mlp.combined <- rbind(performance.mlp.combined, performance.data)
}
head(performance.mlp.combined)

```

```

##   hidden.structure hidden.size hidden.depth err.fct act.fct.name algorithm
## 1           3,5,4           12           3    sse         tanh    rprop+
## 2           3,3,2            8           3    sse         tanh    rprop+
## 3           5,2,4           11           3    sse         tanh    rprop+
## 4           3,1,1            5           3    sse         tanh    rprop+
## 5                3            3           1    sse         tanh    rprop+
## 6           5,3,1            9           3    sse         tanh    rprop+
##   baseline accuracy accuracy.pvalue precision recall   f1 test.n
## 1    0.79    0.93    1.435439e-11    0.83    0.81 0.82    339
## 2    0.79    0.93    1.232907e-12    0.82    0.86 0.84    339
## 3    0.79    0.93    1.232907e-12    0.81    0.89 0.84    339
## 4    0.79    0.92    4.176839e-10    0.75    0.90 0.82    339
## 5    0.79    0.93    1.232907e-12    0.81    0.89 0.84    339
## 6    0.79    0.93    4.299527e-12    0.80    0.87 0.84    339
##   accuracy.pvalue.corrected comparison
## 1    9.617442e-10 Aurignacien vs. Cuneiform (Uruk III)
## 2    8.260476e-11 Aurignacien vs. Cuneiform (Uruk III)
## 3    8.260476e-11 Aurignacien vs. Cuneiform (Uruk III)
## 4    2.798482e-08 Aurignacien vs. Cuneiform (Uruk III)
## 5    8.260476e-11 Aurignacien vs. Cuneiform (Uruk III)
## 6    2.880683e-10 Aurignacien vs. Cuneiform (Uruk III)

```

Density Plots

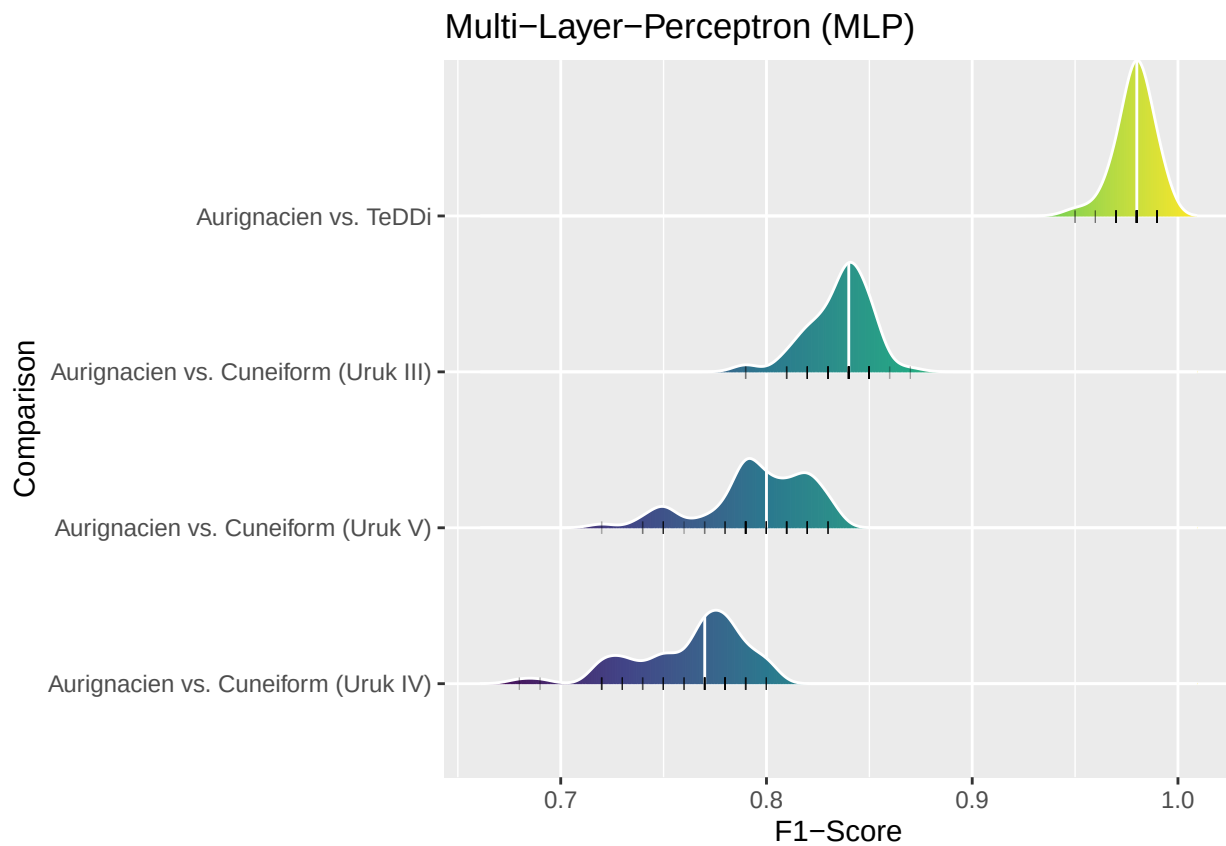
Density distributions of accuracies and F1-scores by comparison. Note: the ordering of “comparisons” according to either F1-scores or accuracies should be consistent across the panels.

F1 Distributions

Distributions of F1-scores by comparison.

```
f1.mlp.plot <- ggplot(performance.mlp.combined, aes(x = f1,
                                                    y = fct_reorder(comparison, f1),
                                                    fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                              quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "F1-Score", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  xlab("F1-Score") +
  ylab("Comparison") +
  theme(legend.position = "none") +
  ggtitle("Multi-Layer-Perceptron (MLP)")
f1.mlp.plot
```

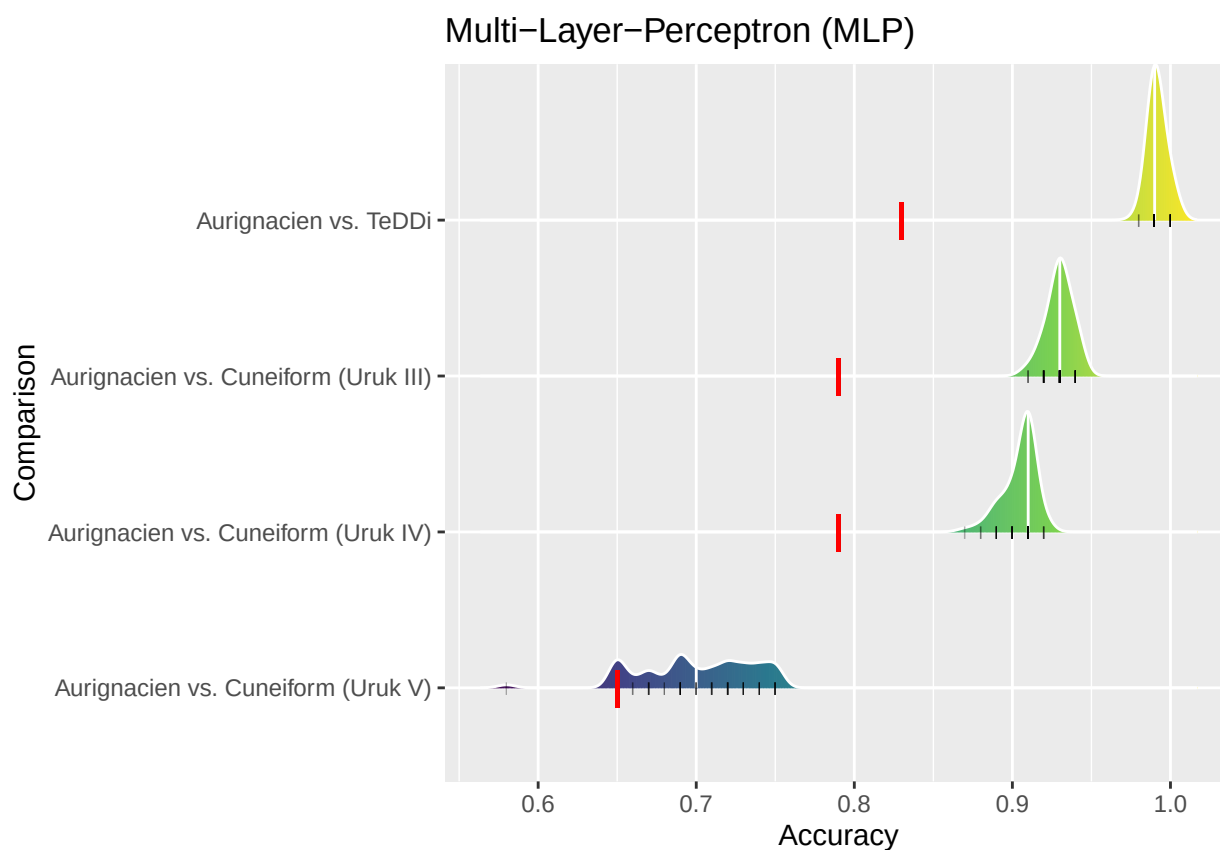
Picking joint bandwidth of 0.0063



Accuracy Distributions

```
accuracy.mlp.plot <- ggplot(performance.mlp.combined, aes(x = accuracy,  
                                                         y = fct_reorder(comparison, accuracy),  
                                                         fill = after_stat(x))) +  
  geom_density_ridges_gradient(scale = 1, color = "white",  
                              quantile_lines = T, quantiles = 2) +  
  scale_fill_viridis_c(name = "Accuracy", option = "D") +  
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +  
  geom_point(shape = 124, size = 5, aes(x = baseline, y = comparison), color = "red") +  
  xlab("Accuracy") +  
  ylab("Comparison") +  
  theme(legend.position = "none") +  
  ggtitle("Multi-Layer-Perceptron (MLP)")  
accuracy.mlp.plot
```

Picking joint bandwidth of 0.00546

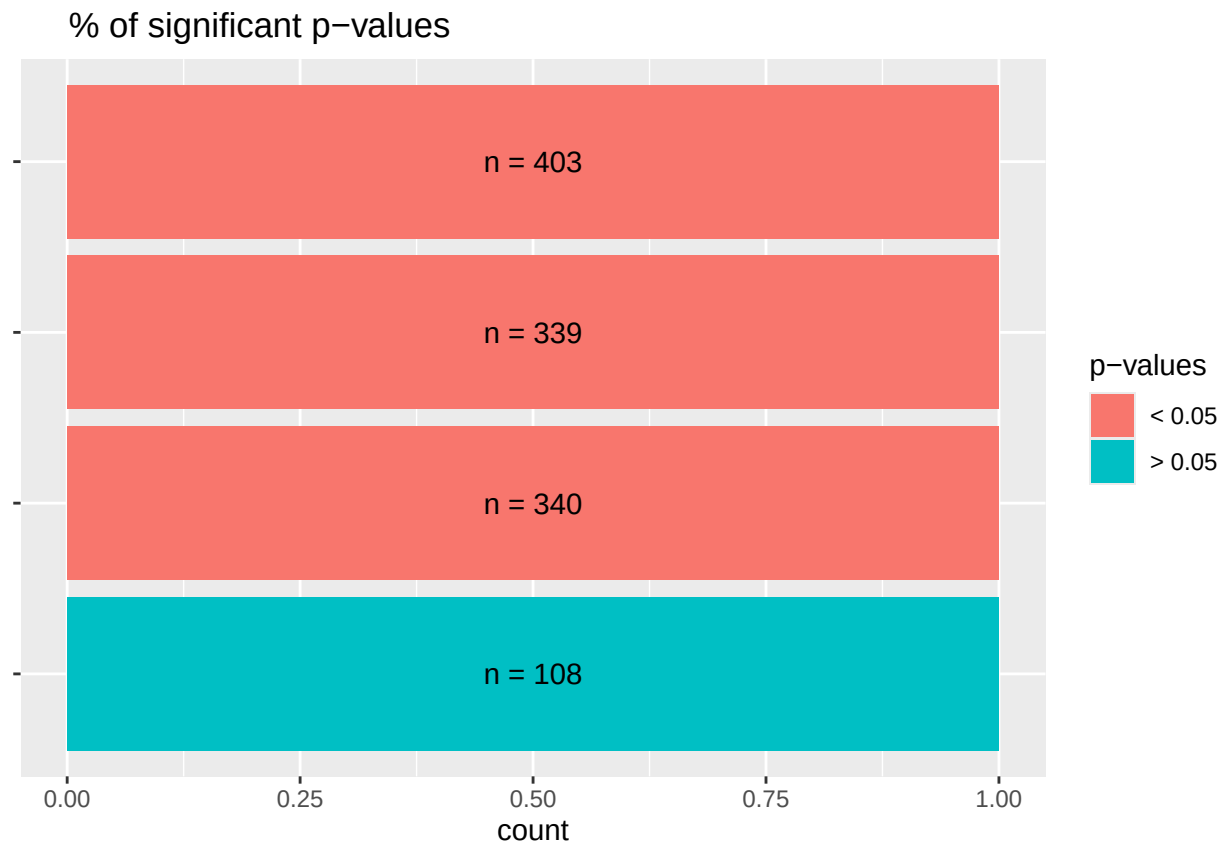


P-value Distributions

Distributions of p-values by comparison.

```
# get split between cases where p < 0.05 and p > 0.05  
performance.mlp.combined$accuracy.pvalue.corrected <-  
  ifelse(performance.mlp.combined$accuracy.pvalue.corrected < 0.05, "< 0.05", "> 0.05")
```

```
# create plot
pvalue.mlp.plot <- ggplot(performance.mlp.combined, aes(y = fct_reorder(comparison, accuracy),
                                                                    fill = as.factor(accuracy.pvalue.corrected))) +
  geom_bar(stat = "count", position = "fill") +
  geom_text(x = 0.5, label = paste("n = ", performance.mlp.combined$test.n, sep = ""),
            check_overlap = T) +
  theme(axis.title.y = element_blank(),
        axis.text.y = element_blank()) +
  scale_fill_discrete(name = "p-values") +
  ggtitle("% of significant p-values")
pvalue.mlp.plot
```



K-Nearest-Neighbors (KNN)

Load Data

```
# list files in the performance folders
files.knn.performance <- list.files(path = "~/Github/PaleoSigns/results/classifier/KNN/performance", full = TRUE)
```

Preprocessing

```
# initialize empty data frame
performance.knn.combined <- data.frame(k = numeric(0),
```



```

        baseline = numeric(0),
        accuracy = numeric(0),
        accuracy.pvalue = numeric(0),
        precision = numeric(0),
        recall = numeric(0),
        f1 = numeric(0),
        accuracy.pvalue.corrected = numeric(0),
        comparison = character(0))

# loop through all files and combine them
for (file in files.knn.performance){
  performance.data <- read.csv(file)
  # apply bonferroni correction
  performance.data$accuracy.pvalue.corrected <- p.adjust(performance.data$accuracy.pvalue,
    method = "bonferroni",
    n = nrow(performance.data))
  # get the subcorpora involved in the binary classification
  filename.parts <- unlist(strsplit(basename(file), "\\_|\\."))
  subcorpus1 <- filename.parts[3]
  subcorpus2 <- filename.parts[4]
  performance.data$comparison <- rep(paste(subcorpus1, "vs.", subcorpus2),
    nrow(performance.data))

  # add to data frame
  performance.knn.combined <- rbind(performance.knn.combined, performance.data)
}
head(performance.knn.combined)

```

```

##   k baseline accuracy accuracy.pvalue precision recall   f1 test.n
## 1 1      0.79      0.93   1.232907e-12      0.81   0.87 0.84   339
## 2 2      0.79      0.93   1.435439e-11      0.83   0.81 0.82   339
## 3 3      0.79      0.93   1.435439e-11      0.82   0.83 0.82   339
## 4 4      0.79      0.92   1.412783e-10      0.79   0.83 0.81   339
## 5 5      0.79      0.92   1.412783e-10      0.79   0.84 0.81   339
## 6 6      0.79      0.92   4.595375e-11      0.81   0.83 0.82   339
##   accuracy.pvalue.corrected comparison
## 1      1.232907e-11 Aurignacien vs. Cuneiform (Uruk III)
## 2      1.435439e-10 Aurignacien vs. Cuneiform (Uruk III)
## 3      1.435439e-10 Aurignacien vs. Cuneiform (Uruk III)
## 4      1.412783e-09 Aurignacien vs. Cuneiform (Uruk III)
## 5      1.412783e-09 Aurignacien vs. Cuneiform (Uruk III)
## 6      4.595375e-10 Aurignacien vs. Cuneiform (Uruk III)

```

Density Plots

F1 Distributions

Distributions of F1-scores by comparison.

```

f1.knn.plot <- ggplot(performance.knn.combined, aes(x = f1,
  y = fct_reorder(comparison, f1),
  fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
    quantile_lines = T, quantiles = 2) +

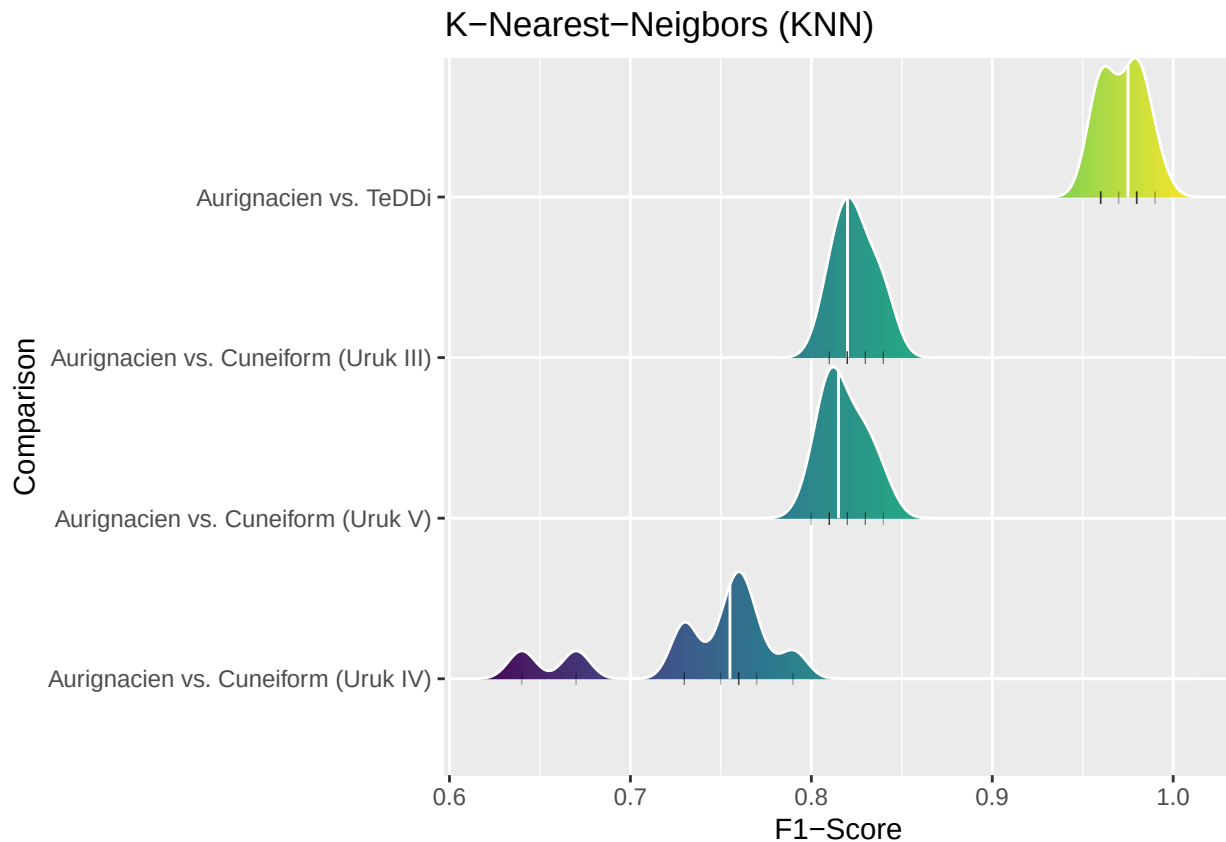
```

```

scale_fill_viridis_c(name = "F1-Score", option = "D") +
geom_point(shape = 124, alpha = 0.4, size = 1.5) +
xlab("F1-Score") +
ylab("Comparison") +
theme(legend.position = "none")+
ggtitle("K-Nearest-Neighbors (KNN)")
f1.knn.plot

```

Picking joint bandwidth of 0.00759



Accuracy Distributions

Distributions of accuracies by comparison.

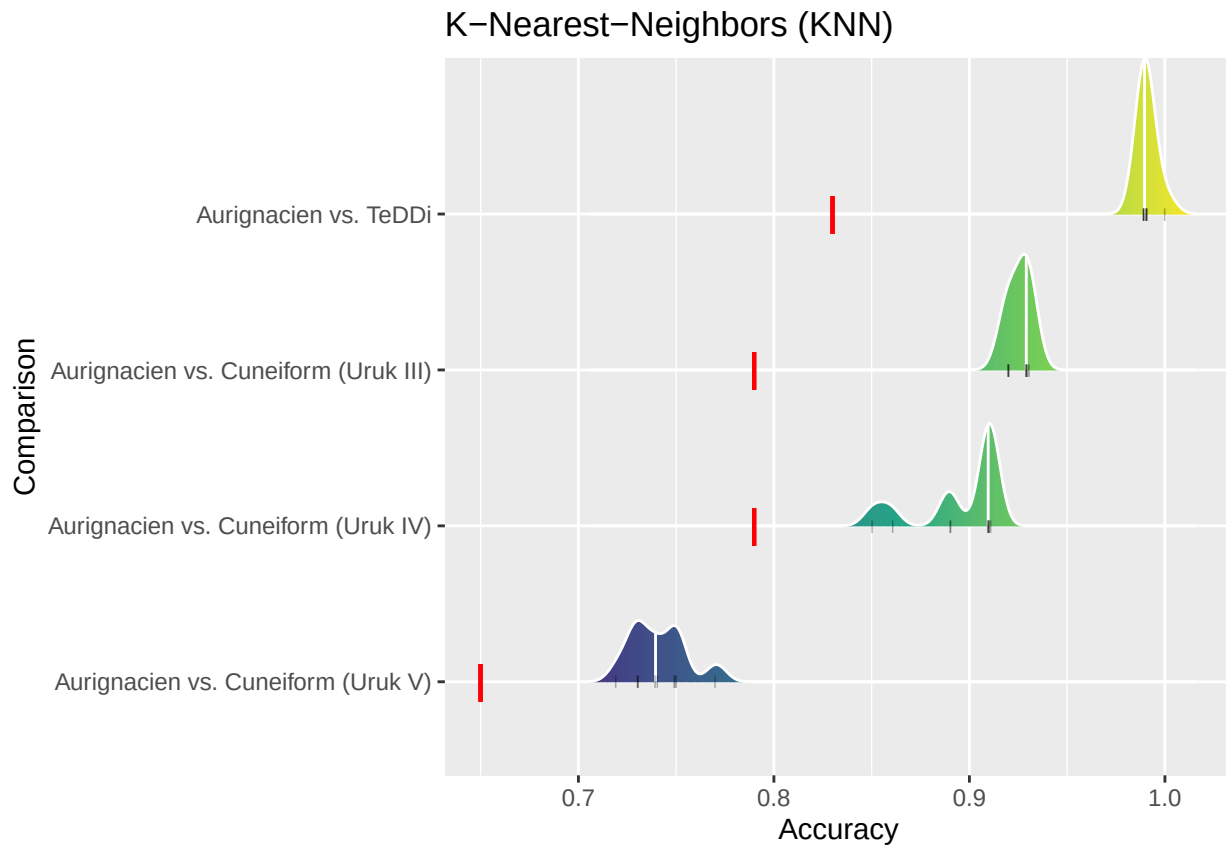
```

accuracy.knn.plot <- ggplot(performance.knn.combined, aes(x = jitter(accuracy, amount = 0.001),
                                                           y = fct_reorder(comparison, accuracy),
                                                           fill = after_stat(x))) +
  geom_density_ridges_gradient(scale = 1, color = "white",
                              quantile_lines = T, quantiles = 2) +
  scale_fill_viridis_c(name = "Accuracy", option = "D") +
  geom_point(shape = 124, alpha = 0.4, size = 1.5) +
  geom_point(shape = 124, size = 5, aes(x = baseline, y = comparison), color = "red") +
  xlab("Accuracy") +
  ylab("Comparison") +
  theme(legend.position = "none") +
  ggtitle("K-Nearest-Neighbors (KNN)")

```

```
accuracy.knn.plot
```

```
## Picking joint bandwidth of 0.00512
```

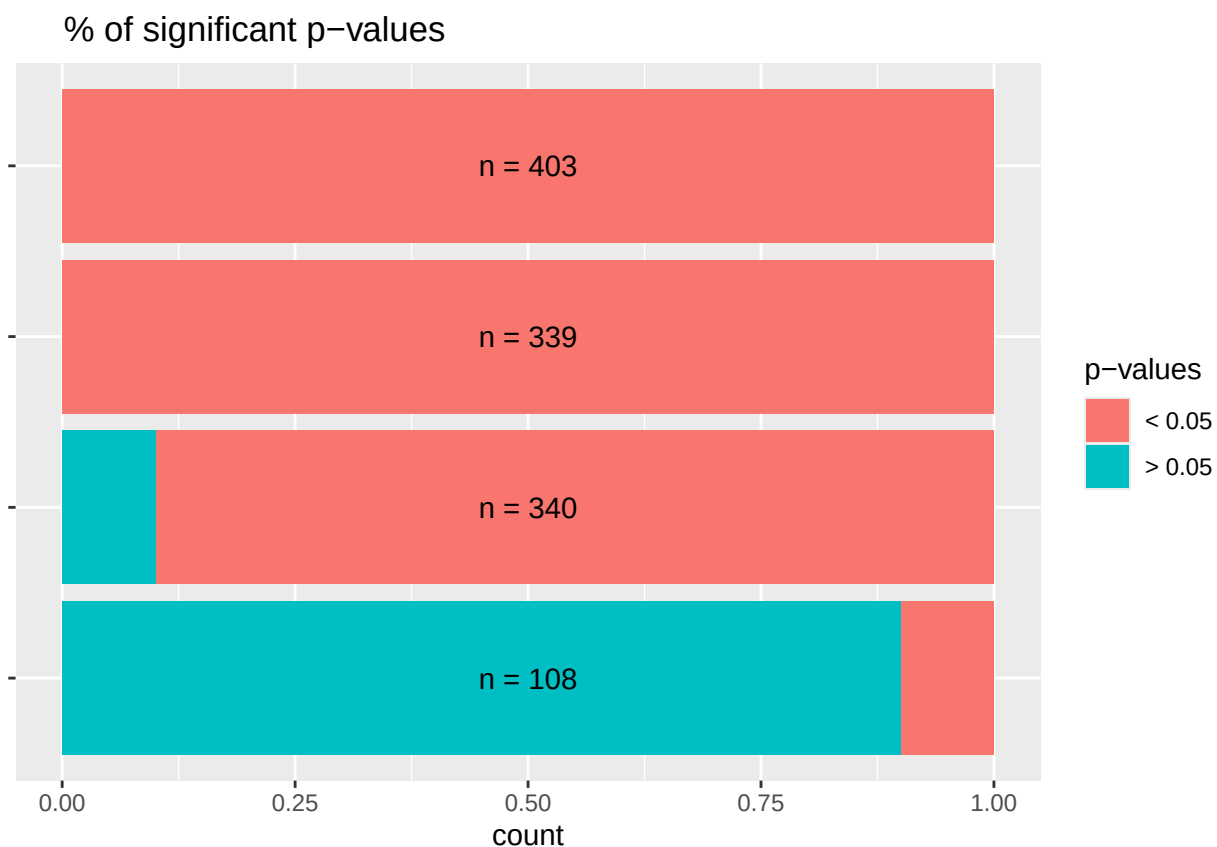


P-value Distributions

Distributions of p-values by comparison.

```
# get split between cases where p < 0.05 and p > 0.05
performance.knn.combined$accuracy.pvalue.corrected <-
  ifelse(performance.knn.combined$accuracy.pvalue.corrected < 0.05, "< 0.05", "> 0.05")

# create plot
pvalue.knn.plot <- ggplot(performance.knn.combined, aes(y = fct_reorder(comparison, accuracy),
  fill = as.factor(accuracy.pvalue.corrected))) +
  geom_bar(stat = "count", position = "fill") +
  geom_text(x = 0.5, label = paste("n = ", performance.knn.combined$test.n, sep = ""),
    check_overlap = T) +
  theme(axis.title.y = element_blank(),
    axis.text.y = element_blank()) +
  scale_fill_discrete(name = "p-values") +
  ggtitle("% of significant p-values")
pvalue.knn.plot
```



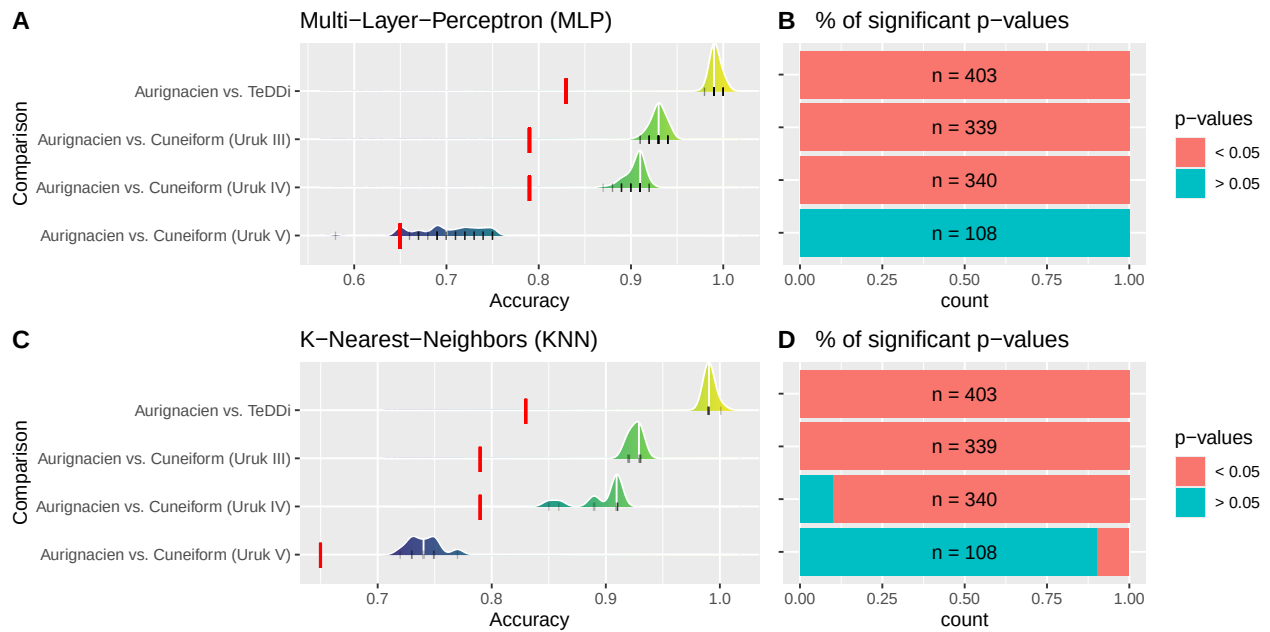
Combine Plots

```
combined.plot <- ggarrange(accuracy.mlp.plot, pvalue.mlp.plot,
                           accuracy.knn.plot, pvalue.knn.plot,
                           ncol = 2, nrow = 2,
                           widths = c(1.5, 1), labels = "AUTO")
```

```
## Picking joint bandwidth of 0.00546
```

```
## Picking joint bandwidth of 0.00507
```

```
combined.plot
```



Safe to file.

```
ggsave("~/Github/PaleoSigns/figures/performancePlot.pdf",
        combined.plot, dpi = 300, width = 10, height = 5, device = cairo_pdf)
```

Package References

```
## Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_.
## Springer-Verlag New York. ISBN 978-3-319-24277-4,
## <https://ggplot2.tidyverse.org>.

## Attali D, Baker C (2023). _ggExtra: Add Marginal Histograms to
## 'ggplot2', and More 'ggplot2' Enhancements_. R package version 0.10.1,
## <https://CRAN.R-project.org/package=ggExtra>.

## Auguie B (2017). _gridExtra: Miscellaneous Functions for "Grid"
## Graphics_. R package version 2.3,
## <https://CRAN.R-project.org/package=gridExtra>.

## Kassambara A (2023). _ggpubr: 'ggplot2' Based Publication Ready Plots_.
## R package version 0.6.0, <https://CRAN.R-project.org/package=ggpubr>.

## Wilke C (2024). _ggribes: Ridgeline Plots in 'ggplot2'_. R package
## version 0.5.6, <https://CRAN.R-project.org/package=ggribes>.

## Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R,
## Golemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E,
## Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi
## K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the
## tidyverse." _Journal of Open Source Software_, *4*(43), 1686.
## doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.

## Barrett T, Dowle M, Srinivasan A, Gorecki J, Chirico M, Hocking T
## (2024). _data.table: Extension of `data.frame`_. R package version
## 1.15.4, <https://CRAN.R-project.org/package=data.table>.
```

```
## Kuhn, Max (2008). "Building Predictive Models in R Using the caret
## Package." _Journal of Statistical Software_, *28*(5), 1-26.
## doi:10.18637/jss.v028.i05 <https://doi.org/10.18637/jss.v028.i05>,
## <https://www.jstatsoft.org/index.php/jss/article/view/v028i05>.
```

Literature References

Hollander, M., Wolfe, D. A., & Chicken, E. (2014). *Nonparametric statistical methods*. John Wiley & Sons.

Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*, 28, 1-26.

McDonald, J.H. (2014). *Handbook of Biological Statistics* (3rd ed.). Sparky House Publishing, Baltimore, Maryland. online at <http://www.biostathandbook.com>
